Chapitre 0 — Avant tout | Partie E — La croix : le joueur | leçon 0.68.4 sur 153 | difficulté 0 / 10

0.68.4. La fonction lire() — lire une case

#include <lire> : lire(colonne, ligne) rend la tuile posée dans cette case — de quoi savoir si elle est libre.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `lire(colonne, ligne)` rend le **numéro de la tuile** affichée dans cette case. Une case vide rend **0**. C'est ainsi qu'un jeu sait s'il y a un mur devant le joueur, sans tenir un tableau à côté.

La ligne à écrire : `#include <lire>`.

Ses arguments : la colonne et la ligne.

Ce qu'elle coûte : environ **90 octets** la première fois.

Essaie : lis la case (4, 3), qui est vide : `t` vaut 0, et le programme écrit NON.

> **On retrouve lire dans 43 leçons**
>
> 0.69, 0.69.1, 0.69.2, 0.69.3, 0.82, 0.82.1, 0.83.2, 0.93.1, 0.93.2, 0.93.3, 0.93.4, 0.93.5, 0.93.6, 0.93.7, 0.93.8, 0.93.9, 0.94, 0.94.1, 0.95, 0.95.1, 0.99, 0.99.1, 0.100, 0.100.1, 0.101, 0.101.1, 0.102, 0.102.1, 0.103, 0.104, 0.105, 0.105.1, 0.106, 0.107, 0.107.1, 0.108, 31, 36, 112, 113, 115, 122, 132

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **else (sinon)** — Ce que l'on fait quand la condition du « if » est fausse.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <lire> : lire une case ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <lire> : lire une case ---- »). La console l'ignore.
`#include <poser>`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer.
`#include <ALPHABET>`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer.
`#include <texte>`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer.
`#include <lire>  // lire() : la tuile d'une case`	Demande à la console « lire » (ce que c'est : lit la tuile posée sur une case). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « lire() : la tuile d'une case ».)
`uint8_t t = 0;`	Une variable : une boîte appelée t, qui retient un nombre de 0 à 255, et qui commence à 0.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`poser(3, 3, ALPHABET[0]);  // un A en (3, 3)`	Appel de la fonction poser de la console, avec les arguments 3, 3, ALPHABET[0]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « un A en (3, 3) ».)
`t = lire(3, 3);  // t reçoit le numéro de la tuile : 1, celui du A`	t prend une nouvelle valeur : lire(3, 3). (Le commentaire le dit : « t reçoit le numéro de la tuile : 1, celui du A ».)
`if (t == ALPHABET[0]) {  // est-ce un A ?`	Une décision : si t est égal à ALPHABET[0], on fait ce qui est entre les accolades. Sinon, on passe. (Le commentaire le dit : « est-ce un A ? ».)
`texte(3, 5, "OUI");`	Appel de la fonction texte de la console, avec les arguments 3, 5, "OUI". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`} else {`	Sinon (si la condition d'avant était fausse), on fait le bloc qui suit.
`texte(3, 5, "NON");`	Appel de la fonction texte de la console, avec les arguments 3, 5, "NON". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().

`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **335 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        52 o  ← la console, à l'allumage  — la mise en route de la console
  main()           51 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte      10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles     58 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles           80 o  ← CopierTuiles  — 5 lettres sur 43
  "OUI"             3 o  ← texte()  — un texte gravé
  "NON"             3 o  ← texte()  — un texte gravé
Lettres gravées : AINOU
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte() × 2    22 o sur place, dans main()  + EcrireTexte (10 o)
  poser() × 1     6 o sur place, dans main()
  lire()  × 1     4 o sur place, dans main()
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <lire> : lire une case ----

#include <poser>
#include <ALPHABET>
#include <texte>
#include <lire>       // lire() : la tuile d'une case
```

2. UNE VARIABLE — T

```
uint8_t t = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  poser(3, 3, ALPHABET[0]);    // un A en (3, 3)
  t = lire(3, 3);              // t reçoit le numéro de la tuile : 1, celui du A

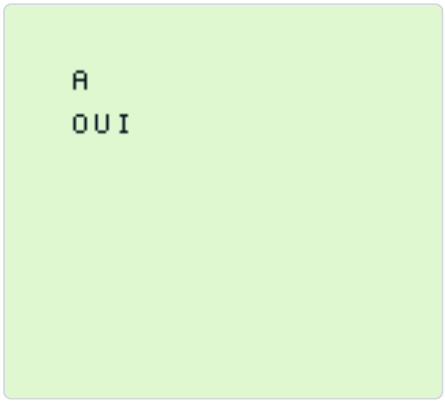
  if (t == ALPHABET[0]) {      // est-ce un A ?
    texte(3, 5, "OUI");
  } else {
    texte(3, 5, "NON");
  }

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Un A, et OUI en dessous : la case lue porte bien un A.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ t vaut 1, le numéro du A (1)
- ✓ le programme a écrit OUI

Ensuite : **0.69. Chercher le A sur l'écran : lire** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++